

Q1 Student Management System [BL3 – Apply]

10 marks

Write pseudocode to design a Student class with attributes (name, rollNo, marks) and methods to:

- Accept student details
- Calculate average marks
- Display result (Pass/Fail)

Apply encapsulation and data hiding by restricting direct access to attributes.

OOP Concepts: Encapsulation | Data Hiding | Classes & Objects

Your Answer

```
Start
class Student
Private : name , roll.no, marks

    Method AcceptStudentDetails()
    {
        input name , roll.no , marks
    }
    Method CalculateAverage()
    {
        average=marks
        Return average
    }
```

Method AcceptStudentDetails()

```
{  
  input name , roll.no , marks  
}
```

Method CalculateAverage()

```
{  
  average=marks  
  Return average  
}
```

Method DisplayResults()

```
{  
  If (marks > 50 )  
    print "Pass"  
  Else  
    print "Fail"
```

Main

```
Create Student object  
Call AcceptStudentDetails()  
Call CalculateAverage()  
Call DisplayResult()
```

Stop

 Save Answer

Q2 2. Library Book Management [BL3 – Apply]

10 marks

Design a Book class with private attributes (title, author, ISBN, isAvailable). Implement methods to:

- Issue a book (change availability status)
- Return a book
- Display book details using getter methods

Demonstrate object creation for at least 3 books and simulate issue/return operations.

OOP Concepts: Encapsulation | Getters & Setters | Objects

Your Answer

Start

Class Book

Private : title, author, ISBN, isAvailable

Method IssueBook()

```
{  
    isAvailable = False  
}
```

Method ReturnBook()

```
{  
    isAvailable = True  
}
```

Method DisplayBook()

```
Method DisplayBook()
{
    Print tile , author , ISBN , isAvailable
}
```

Main

```
Create book b1 , b2 , b3
Call IssueBook() for b1
Call ReturnBook() for b1
Call DisplayBook() for b1
Call DisplayBook() for b2
Call DisplayBook() for b3
```

Stop

 Save Answer

Q3 Employee Salary Calculation [BL3 – Apply]

10 marks

Write pseudocode for an Employee class with encapsulated attributes (empId, name, basicPay, designation). Implement methods to: •

Calculate gross salary (basic + HRA + DA)

- Calculate net salary after deductions (PF, tax)

- Display pay slip

Apply data hiding so salary components are not directly accessible.

OOP Concepts: Encapsulation | Data Hiding | Methods

Your Answer

```
Start
Class Employee
Private : empId , name , basicPay , designation

    Method Calculategross()
    {
        Gross = basicPay + HRA + DA
    }
    Method CalculateNet()
    {
        Net = Gross - PF-tax
    }
```

Your Answer

```
Gross = basicPay + HRA + DA
}
Method CalculateNet()
{
    Net = Gross - PF-tax
}
Method DisplayPaySlip()
{
    Print empld , name , designation
    Print Gross , Net
```

Main

```
Create Employee object
Call CalculateGross()
Call CalculateNet()
Call DisplayPaySlip()
```

Stop

 Save Answer

Q4 Shape Area Calculator using Polymorphism [BL3 – Apply]

10 marks

Create a Shape base class with an overridable calculateArea() method. Apply this to:

- Circle — area using radius
- Rectangle — area using length and breadth
- Triangle — area using base and height

Use method overriding to demonstrate runtime polymorphism. Call calculateArea() for each object.

OOP Concepts: Inheritance | Polymorphism | Method Overriding

Your Answer

Start

Class Shape

Method CalculateArea()

Class Circle extends Shape

Override CalculateArea()

area = $3.14 * r * r$

Class Rectangle extends Shape

Override CalculateArea()

area = length * breadth

Class Rectangle extends Shape

Override CalculateArea()

area = length * breadth

Class Triangle extends Shape

Override CalculateArea()

area = 0.5 * base * height

Main

Create Circle object

Create Rectangle object

Create Triangle object

Call CalculateArea() for each object

Stop

 Save Answer

Q5 Vehicle Registration System [BL3 – Apply]

10 marks

Design a Vehicle class with attributes (regNo, owner, fuelType). Extend it into:

- TwoWheeler — with engine capacity
- FourWheeler — with seating capacity and AC status

Apply inheritance to reuse common attributes and override a displayDetails() method in each subclass.

OOP Concepts: Inheritance | Method Overriding | Subclass

Your Answer

Start

Class Vehicle

regNo , owner , fuelType

Method DisplayDetails()

Class TwoWheeler extends Vehicle

engineCapacity

Override DisplayDetails()

Print regNo , owner , fuelType , engineCapacity

Class FourWheeler extends Vehicle

seatingCapacity , ACStatus

Class FourWheeler extends Vehicle
seatingCapacity , ACStatus

Override DisplayDetails()
Print regNo , owner , fuelType , seatingCapacity , ACStatus

Main
Create TwoWheeler object
Create FourWheeler object
Call DisplayDetails() for both objects
Stop

 Save Answer

Q6 Bank Account Hierarchy [BL4 – Analyze]

10 marks

Develop pseudocode for a BankAccount superclass and two subclasses — SavingsAccount (with interest calculation) and CurrentAccount (with overdraft facility). Analyze and implement:

- Inheritance of common attributes (accNo, balance, holder)
- Method overriding for withdrawal behavior
- How overdraft protection differs from savings limit

Compare the behavior of withdraw() across both subclasses and justify the design choices.

OOP Concepts: Inheritance | Method Overriding | Polymorphism

Your Answer

Start

Class BankAccount

accNo , holder , balance

Method Withdraw()

Class SavingsAccount extends BankAccount

interestRate

Override Withdraw()

If balance >= amount

balance = balance - amount

Your Answer

```
Override Withdraw()  
    If balance >= amount  
        balance = balance - amount  
    Else  
        Print "Insufficient Balance "  
  
Class CurrentAccount extends BankAccount  
    overdraftLimit  
    Override Withdraw()  
        If balance + overdraftLimit >= amount  
            balance = balance - amount  
        Else  
            Print "Overdraft Limit Exceeded"  
  
Main  
    Create SavingsAccount object  
    Create CurrentAccount object  
    Call Withdraw() for both objects  
  
Stop
```

 Save Answer

Q7 Hospital Patient Record System [BL4 – Analyze]

10 marks

Design a Patient base class and analyze how it extends into:

- InPatient — with ward number, admission date, and daily charges
- OutPatient — with appointment date and consultation fee

Analyze encapsulation requirements for sensitive data (diagnosis, prescription). Override a generateBill() method for each type and compare the billing logic differences. OOP Concepts: Encapsulation | Inheritance | Polymorphism

Your Answer

Start

Class Patient

Private : patientId , name , diagnosis , prescription

Method GenerateBill()

Class InPatient extends Patient

wardno , admissionDate , dailycharges

Override GenerateBill()

bill = dailycharges * days

Print bill

```
Class Outpatient extends Patient
    appointmentDate , consultationFee
    Override GenerateBill()
        bill = consultationFee
        Print bill
```

```
Main
    Create InPatient object
    Create OutPatient object
    Call GenerateBill() for both objects
Stop
```

 Save Answer

Q8 E-Commerce Product Catalog [BL4 – Analyze]

10 marks

Analyze and design a Product class hierarchy for an e-commerce system:

- Electronics — with warranty, brand, voltage
- Clothing — with size, fabric, gender
- Grocery — with expiryDate and weight

Override an applyDiscount() method for each category with different discount rules. Analyze why polymorphism makes the cart checkout process flexible and extensible. OOP Concepts: Polymorphism | Inheritance | Method Overriding

Your Answer

Start

Class Product

Method ApplyDiscount()

Class Electronics extends Product

warranty , brand , voltage

Override ApplyDiscount()

Calculate electronics discount

Class Clothing extends Product
size , fabric , gender
Override ApplyDiscount()
Calculate clothing discount

Class Grocery extends Product
expiryDate , weight
Override ApplyDiscount()
Calculate grocery discount

Main

Create Electronics object
Create Clothing object
Create Grocery object
Call ApplyDiscount() for each object

 Save Answer

Q9 University Staff Hierarchy [BL4 – Analyze]

10 marks

Analyze the design of a Staff base class extended by TeachingStaff and NonTeachingStaff. Further extend TeachingStaff into Professor and Lecturer (multilevel inheritance). For each level:

- Identify which attributes should be private vs protected
- Override calculateAllowance() at each level
- Analyze how protected access enables inheritance without full exposure Justify the use of multilevel inheritance and explain how access modifiers enforce data hiding.

OOP Concepts: Multilevel Inheritance | Access Modifiers | Data Hiding

Your Answer

Start

Class Staff

Protected : name , id

Method CalculatedAllowance()

Class TeachingStaff extends Staff

Override CalculateAllowance()

Class Professor extends TeachingStaff

Override CalculateAllowance()

Class Professor extends TeachingStaff
Override CalculateAllowance()

Class Lecturer extends TeachingStaff
Override CalculateAllowance()

Class NonTeachingStaff extends Staff
Override CalculateAllowance()

Main

Create Professor object

Create Lecturer object

Create NonTeachingStaff object

Call CalculateAllowance() for each object

Stop

 Save Answer

Q10 Smart Home Device Controller [BL4 – Analyze]

10 marks

Analyze and design a SmartDevice superclass with subclasses: SmartLight, SmartThermostat, and SmartLock. Override an executeCommand(cmd) method in each, where the same command (e.g., 'on', 'off', 'status') produces device-specific behavior. Analyze:

- How polymorphism allows a single controller loop to manage all devices
 - Security implications of data hiding in SmartLock (PIN must never be exposed)
- Design the class hierarchy and explain the OOP principles applied at each level.

OOP Concepts: Polymorphism | Encapsulation | Inheritance

Your Answer

Start

Class SmartDevice

Method ExecuteCommand(cmd)

Class SmartLight extends SmartDevice

Override ExecuteCommand(cmd)

Class SmartThermostat extends SmartDevice

Override ExecuteCommand(cmd)

Your Answer

```
Class SmartLight extends SmartDevice  
    Override ExecuteCommand(cmd)
```

```
Class SmartThermostat extends SmartDevice  
    Override ExecuteCommand(cmd)
```

```
Class SmartLock extends SmartDevice  
    Private : PIN  
    Override ExecuteCommand(cmd)
```

Main

```
Create SmartLight object  
Create SmartThermostat object  
Create SmartLock object  
Call ExecuteCommand("ON")  
Call ExecuteCommand("OFF")  
Call ExecutedCommand("STATUS")
```

 Save Answer